

Generative AI: retrieval and evaluation

AI ENGINEERING: GENAI AND AGENTIC SYSTEMS

Today you build `nw/policy`, Project 3: a policy service that answers from the current corpus, cites the passage, refuses when it should, and proves it with a harness. On the platform its prompt is a versioned record with an approval, its chunks live in your tenant's vector store, and every model call goes through the model gateway on your key.

An LLM predicts tokens, so facts and currency are your job

What it is

- A model that predicts the next token (a word, a subword or a character), then the next, from everything before it
- General: one model drafts, summarises, extracts and classifies
- Steered by the prompt and by the context you put in front of it

What it is not

- A database: it keeps no record of which document said what
- Current: it knows its training data, not this quarter's SLA
- A source you can cite: a fluent answer and a true one look the same

Reasoning models trade answer-time compute for accuracy

More thinking time

74 vs 12

percent on AIME 2024, one sample: o1 against GPT-4o (OpenAI, 2024)

Compute beats size, sometimes

With compute-optimal thinking at test time, a smaller model can outperform one **14 times larger** at matched compute, on problems it already partly solves.

Now open weights too

DeepSeek-R1 taught reasoning through reinforcement learning alone, then distilled it into small open models. Published in Nature.

For a support desk the trade is latency and tokens on every answer. Spend it on the hard ticket, not on the password reset.

Long context does not replace retrieval; it changes the price

	Long context	Retrieval
Quality, budget no object	higher on average	lower on average
Cost per question	every token, every time	a handful of passages
How it fails	the middle of the window gets lost	the right passage never arrives
Current and citable	whatever you pasted	filtered by metadata, cited by id

Route, do not pick

Windows have grown about 30x a year since 2023, and 1M tokens ships today.

Letting the model choose retrieval or long context per query came close to long-context quality at much lower cost.

An LLM judge agrees with people about as often as people agree, with biases you can name

80%+

agreement of strong LLM judges with human preferences, the same rate at which humans agree with each other

- **Position:** prefers the answer shown first
- **Verbosity:** prefers the longer answer
- **Self-enhancement:** prefers answers from its own model family

The agreement rate was measured on someone else's task. Yours is measured on your labels: today's Judge is calibrated against 30 of them before anyone reads its score.

Models hallucinate because training rewards a confident guess, so design for refusal

Training and benchmarks score a guess above "I do not know", so models learn to guess. The researchers propose scoring abstention instead.

Kalai et al., OpenAI and Georgia Tech, 2025

Even with the source in hand

1.8% to 11%

hallucination rate on grounded summarisation: the best model, then several frontier models (Vectara leaderboard, September 2026)

Today a refusal is a named outcome with a reason, and an answer with no valid citation is refused. That is the engineering answer to a guess.

Prompt injection tops the OWASP list, and no fool-proof fix is known

OWASP LLM 2025	Where Project 3 answers it
1 Prompt Injection	the guardrail on <code>/ask</code> , context marked as data
2 Sensitive Information Disclosure	redaction before indexing, audience filter
3 Supply Chain	locked dependencies, pinned model ids
4 Data and Model Poisoning	corpus hash in the manifest, <code>--check</code>
5 Improper Output Handling	structured output, citations validated

OWASP LLM 2025	Where Project 3 answers it
6 Excessive Agency	the next part: the approval gate
7 System Prompt Leakage	prompts hold no secrets; keys live in the secret store
8 Vector and Embedding Weaknesses	metadata filters on the fused pool
9 Misinformation	refusal, the Judge, the gate
10 Unbounded Consumption	a budget per gateway key

Measured in support, the gain is real; measured on cost alone, it reverses

+14%

issues resolved per hour across 5,179 support agents with a generative AI assistant; +34 percent for novices, minimal for the most experienced

Brynjolfsson, Li and Raymond, The Quarterly Journal of Economics, 2025

The headline, then the correction

February 2024: Klarna said its assistant handled two thirds of its service chats in its first month, the work of 700 full-time agents.

May 2025: its CEO said cost had been "a too predominant evaluation factor", leading to lower quality, and Klarna began hiring people again, as reported from a Bloomberg interview.

Northwind can classify a ticket; nothing yet answers the customer

PART 1

Foundations and
the AI service layer

PART 2

Machine learning in
production

PART 3

Deep learning and
representation

PART 4

Generative AI:
retrieval and
evaluation

PART 5

Agentic AI systems

PART 6

Deployment,
observability and
capstone

Small models chose urgency and category for almost nothing, trained by a pipeline and approved in a registry. A question about the SLA needs a generator held to a source: an LLM with retrieval, citations and a refusal around it. The agent in the next part calls it as `search_policies` .

Today you build grounded answers with citations or a refusal, judged and gated

- 1 Read the corpus and the contract
- 2 Tokenise sensitive fields before indexing
- 3 Split on headings, carry the metadata
- 4 Hybrid retrieval, filters, a reranker
- 5 Citations enforced in code, or refuse
- 6 Prompts in the platform's prompt store
- 7 The harness, and the judge calibrated
- 8 The gate on every pull request, one experiment
- 9 The index in the platform's vector store
- 10 The service through the gateway: cache, feedback
- 11 The service knows when the questions change
- 12 Session check and commit
- 13 Argue with the judge (optional)

28 documents, three superseded by a newer version and four internal. 77 golden cases, 17 of them must refuse.

Sensitive values never reach the index, the store, the prompt or the log

```
"Contact ops@bluefreight.example  
about account NW-11498"
```

```
"Contact [EMAIL_1]  
about account [ACCOUNT_1]"
```

Same value, same token within one text. The model can still say "the customer at [ACCOUNT_1] asked twice".

Redaction runs inside `chunk_document`

Once Step 2 passes, no chunk can carry an email or an account id, and a test proves it end to end. The chunks you upsert in Step 9 are these chunks, so the managed store never sees a raw value either.

The phone pattern requires a country code so that `2025-03-01` survives. Dates and percentages are the answers.

Chunking is an engineering decision, and the index remembers it

Strategy	Keeps	Loses	Use when
Fixed windows	nothing to configure	headings, section boundaries, metadata	unstructured prose
Heading - aware	section, title, metadata per chunk	nothing important on structured docs	policies, runbooks, contracts
Sentence windows with overlap	local context	document structure	long narrative text

A chunk is one heading's text carrying document, section, effective date, audience and `superseded_by`. The build writes `manifest.json`: corpus hash, chunker parameters, both models, about 212 chunks, the prompt versions and a drift baseline. `build_index --check` exits 1 when the corpus has moved on.

Fuse dense and BM25 first, filter second, rerank last

- 1 Dense candidates: paraphrases, but blurs `99.95` and `sla-2023`
- 2 Lexical candidates, fused with `rnf`: each list adds `1 / (60 + rank)`, no weight to tune
- 3 Filter: drop superseded when `current_only`, drop internal when the audience is `customer`
- 4 Cross-encoder reranks the top `max(3k, 10)`
- 5 Return the top `k`

Metadata decides, not luck

A superseded document may be the best lexical match. The filter runs on the fused pool, on the metadata every chunk carries.

The reranker reads query and chunk together: most of today's quality gain, and most of its latency.

Citations are enforced in code, not requested in the prompt

- 1 Tell the model which chunk ids it may cite
- 2 Check every citation against the ids that were actually in the context
- 3 Drop invented ids and count them
- 4 No valid citation left: an answer from memory. Refuse it

Deterministic, so exactly 1.0

Citation validity in the harness must be 1.000. If it is not, the code is wrong, not the model.

Refusal is an outcome, not an error

Path	Reason on the response	Model called
Retrieval too weak to bother the model	<code>weak_retrieval</code>	no
The model says the context does not cover it	<code>model_unanswerable</code>	yes
No valid citation survived the check	<code>no_valid_citation</code>	yes
The track's guardrail blocked the question	<code>screened</code>	no

The duty manager rule for P0 tickets exists only in the internal escalation matrix. A customer asking gets a refusal; the same question with `audience: internal` gets an answer citing `escalation-matrix`.

A prompt is code the model runs, so it gets a version and an approval

```
policy.answer@6c2999977051
```

The name and the first twelve hex characters of the SHA-256 of the text. One changed word, a new hash.

```
ANSWER_PROMPT = register("policy.answer",  
SYSTEM)
```

- The hash rides on every answer, refusals included, every harness row, `eval.json`, the index manifest and the model-call span
- The platform's prompt store keeps the same hash as a version; `register` is idempotent
- `set_stage(..., APPROVED)` is the product owner's decision; `Stage.LIVE` belongs to the capstone
- Git versions the file; the hash versions what the running process sent

Evaluate retrieval and generation separately

Metric	Measures	Computed by
Recall@k, MRR	did the gold chunks make the list, and how high	deterministic
Faithfulness	is every claim supported by the cited passages	the Judge, against a rubric
Citation validity	were the citations real	deterministic, must be 1.0
Refusal correctness	refused when it should, answered when it should	deterministic
Latency, cost	per case	the cost meter, and the gateway's spend per key

A bad answer from good retrieval is a prompt or model problem. A bad answer from bad retrieval is a chunking or index problem. One combined score hides which one you have.

A judge is an instrument, so calibrate it before you read it

0.8

Cohen's kappa on 30 human-labelled answers; false pass rate 0

- A stronger model than the Workhorse, a rubric with a hash, a fixed version: Opus grades gpt-oss, never the other way round
- Agreement about 0.90 at the 0.75 threshold, false fail about 0.19
- Kappa corrects for chance; agreement alone flatters a judge that passes everything
- The false pass rate bounds what the gate can detect

Pay for the judge once, gate for free on every pull request

Run	Calls	Gates on
judged	Workhorse, Judge	five metrics; the baseline
<code>--no-judge</code>	Workhorse	four
<code>--retrieval- only</code>	none	recall@k, MRR

Tolerance 0.03. Citation validity below 1.0 fails always. A baseline from another corpus or prompt is flagged before the verdict.

`eval-gate.yml`, two jobs

retrieval on every pull request that touches the corpus, the package, the prompts or the golden set: build, `--check`, the free harness. No credentials.

gate on demand through OpenID Connect into the platform's account, with the tenant's gateway key.

One variable, one row on the board

Variable	Flag
Chunk size 200	<code>build_index --max-tokens 200</code> , then evaluate
k = 2, 4 or 12	<code>--k 2</code> , <code>--k 4</code> , <code>--k 12</code>
Hybrid off, reranker off	<code>--no-hybrid</code> , <code>--no-rerank</code>
Minimum score 0.2	<code>--min-score 0.2</code>

Retrieval variables first with `--retrieval-only` : eight seconds, nothing spent. Then `--no-judge` for the row: which metric moved, by how much, in which direction, and what it cost in latency.

The store is a platform choice, not a quality one

NW_RETRIEVER	Searches	Track
<code>inprocess</code>	FAISS and BM25 over <code>artifacts/policy</code> , the default	any
<code>knowledge-base</code>	the tenant's Bedrock Knowledge Base on S3 Vectors	AWS
<code>rag-engine</code>	the tenant's RAG Engine corpus	Google Cloud
<code>ai-search</code>	the tenant's Azure AI Search index, hybrid	Azure
<code>qdrant</code>	the tenant's Qdrant collection on :6333	Local

The managed store returns dense candidates. The audience and currency filters and the reranker still run in `retrieval.py`, on the metadata the upsert carried.

Every model call goes through one door that holds the key, the budget and the bill

```
curl -s $NW_GATEWAY_URL/key/info \
-H "Authorization: Bearer $NW_GATEWAY_KEY"
```

`spend`, `max_budget`, `key_alias`: your line on the platform. `GET /spend/keys` with the master key is every tenant's line.

The meter estimates, the gateway bills

`nw_policy_spend_usd_total` is the service's estimate from token counts and the price table. The gateway's `spend` is what the platform charges your tenant.

When the two disagree, explain the difference before you trust either.

The service closes the loop: a cache that cannot lie, feedback that becomes cases, drift that says when to look

Exact-match cache

Key: normalised question, audience, k, index hash, prompt version, model id. A rebuild or a prompt change empties it without a flush.

Not semantic: the Pro SLA is not the Enterprise SLA.

POST /feedback

`helpful`, `wrong` or `unsafe` on an `answer_id`. `feedback --to-golden` prints the wrong ones in the golden set's shape; a person writes the answer.

/drift

Top-hit confidence PSI and refusal rate against the golden baseline of 0.221. Fifty "Login broken" questions: PSI 8.75, refusal rate 1.0, one `drift_alert`, nothing spent.

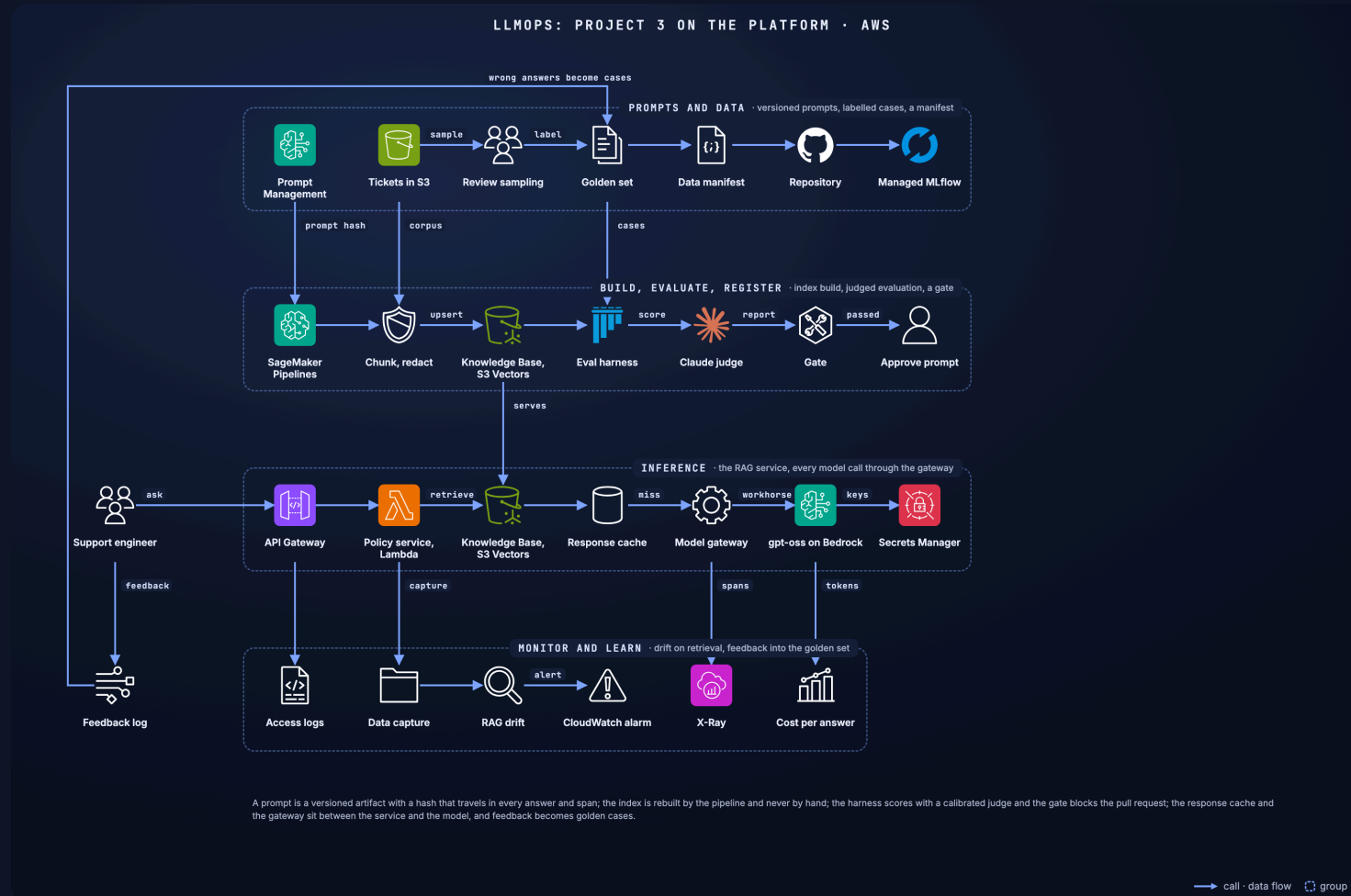
Six things get a version, and none of them is a weights file

Versioned	Where	Changes when
Prompt	<code>name@hash</code> in code; the same hash on the store's version, with its stage	one word of a prompt or rubric
Model id	by role in <code>nw/config.py</code> ; the gateway's model name	the track, a deprecation
Index manifest	corpus hash, chunker, embedder, reranker, prompt versions, drift baseline	the corpus, the chunker, either model
Vector store contents	the tenant's collection, filled from the index	the index
Golden set	77 cases, 17 must refuse, <code>golden_sha256_12</code> in the report	feedback becomes cases
Baseline	one judged run with its prompt versions and corpus hash	a judged run you adopt

One contract per box, one implementation per track

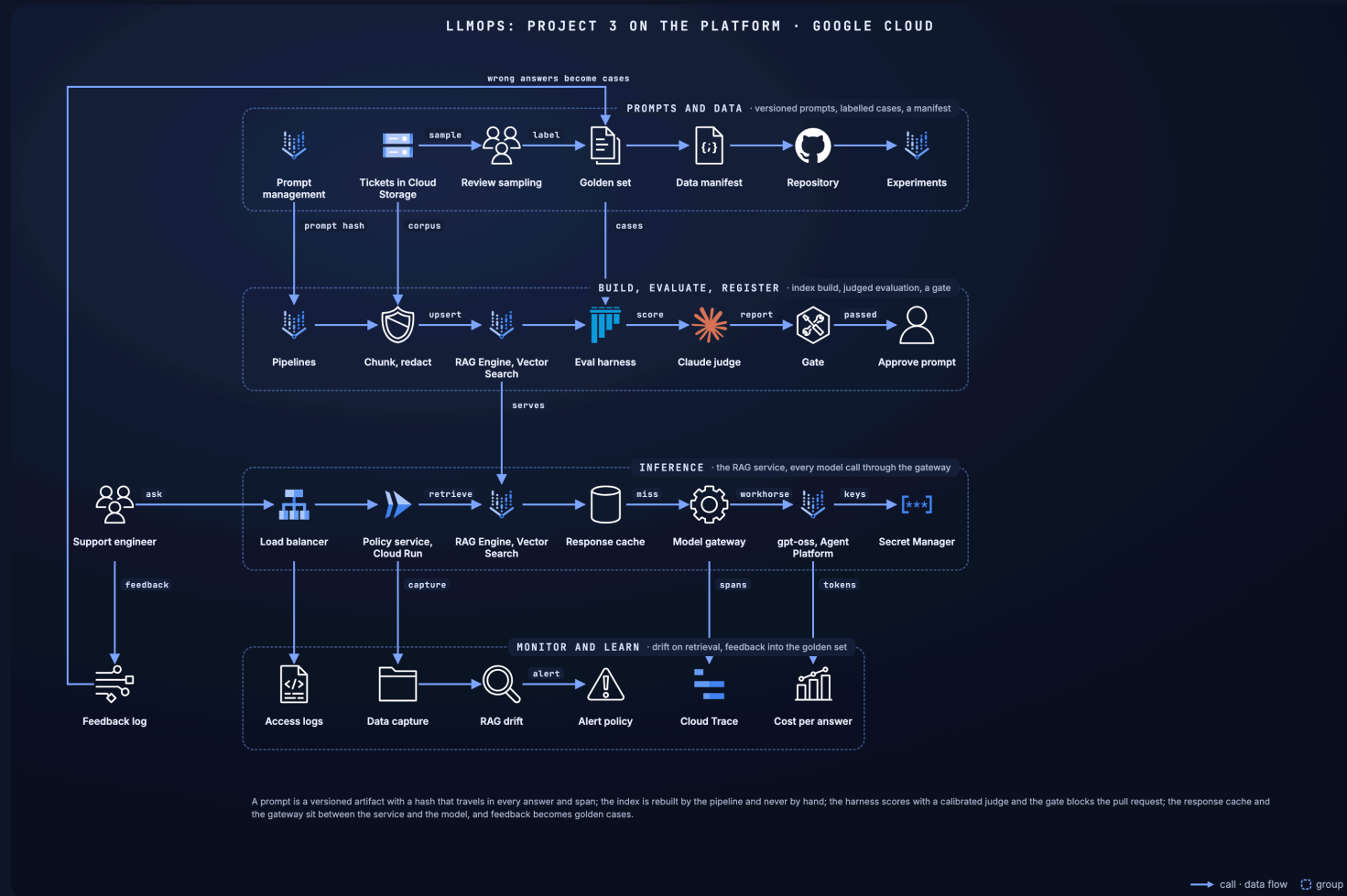
Contract	Local	AWS	Google Cloud	Azure
PromptStore	MLflow prompts	Bedrock Prompt Management	prompt management, stages in the bucket	blobs plus Azure ML data assets by hash
VectorStore	Qdrant	Knowledge Base on S3 Vectors	RAG Engine corpus	Azure AI Search index
Model gateway	LiteLLM	LiteLLM on ECS Fargate	LiteLLM on Cloud Run	API Management AI gateway
Workhorse, Judge	gpt-oss-20b; Claude or judge-free	gpt-oss-120b, Claude Opus 5	gpt-oss-120b, Claude Opus 5	gpt-oss-120b, Claude Opus 5 on Foundry
Screening on /ask	Llama Guard 3	Bedrock Guardrails	Model Armor	Prompt Shields
Drift	Evidently	alarm on drift_alert	log-based metric	log search alert on drift_alert

LLMOps on the platform, AWS track



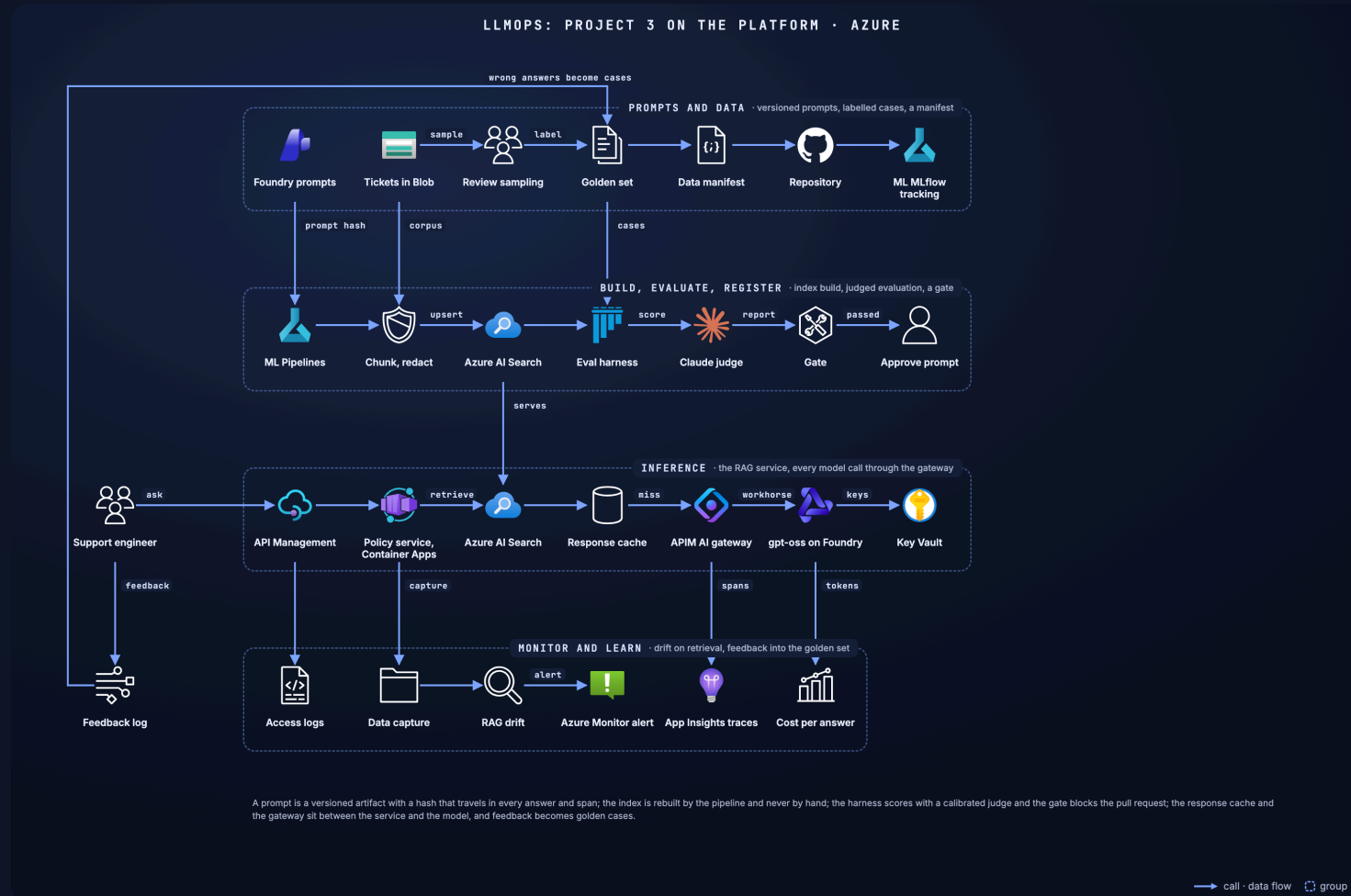
Prompts and data, build and evaluate, inference, monitor and learn: Prompt Management, the Knowledge Base on S3 Vectors, the eval harness with the Claude judge and the gate, the policy service behind API Gateway, the model gateway to gpt-oss on Bedrock, drift and feedback back into the golden set.

LLMOps on the platform, Google Cloud track



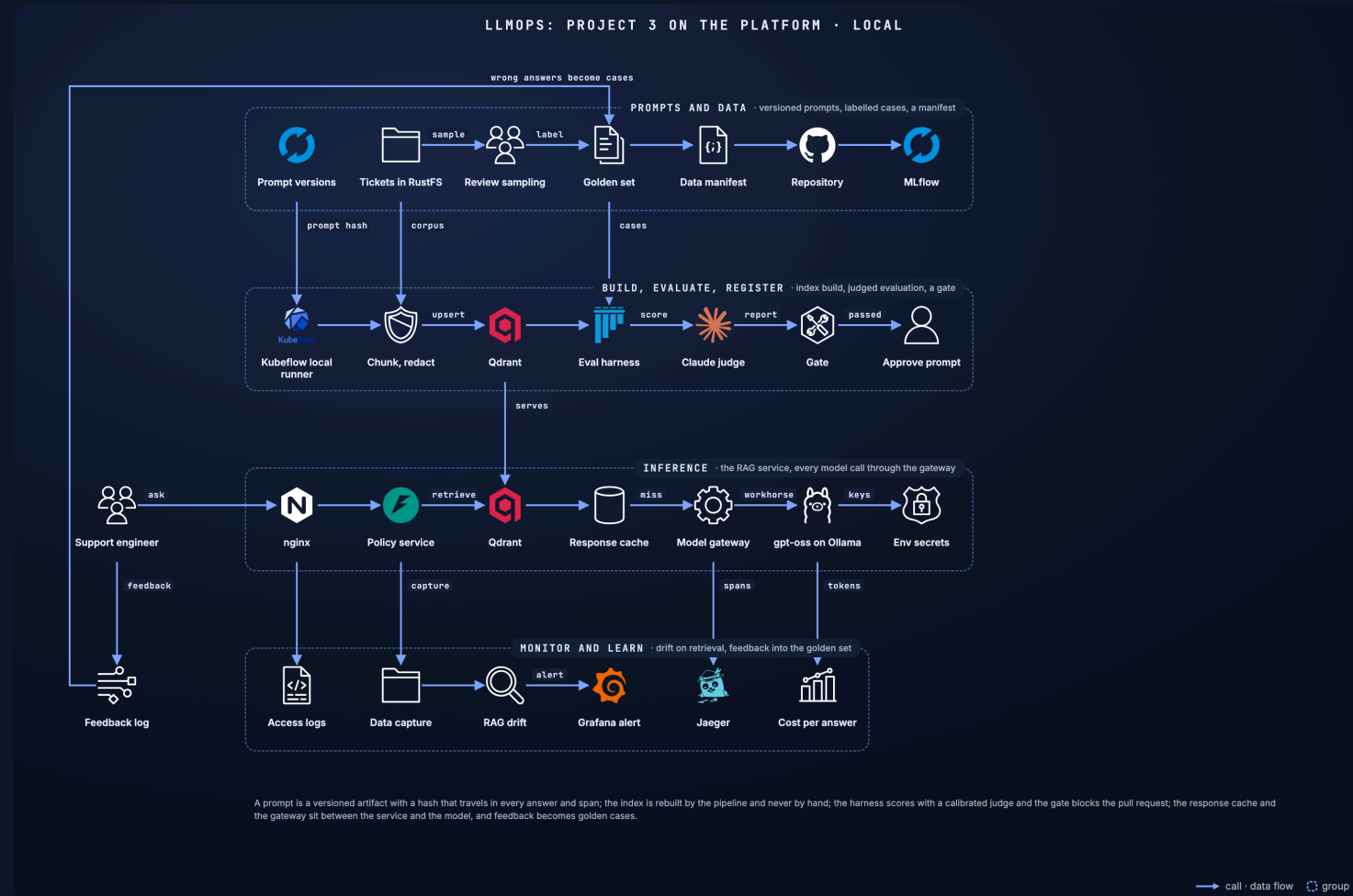
The same loop with prompt management on the Agent Platform, the RAG Engine corpus, gpt-oss through the gateway on Cloud Run, Model Armor and log-based drift.

LLMOps on the platform, Azure track



The same loop on Azure: prompts as blobs plus Azure Machine Learning data assets keyed by the hash, the Azure AI Search index, the policy service on Container Apps, API Management's AI gateway to gpt-oss on Microsoft Foundry, Application Insights traces and cost per answer.

LLMOps on the platform, Local track



The same loop in the compose stack: MLflow prompt versions, Qdrant, the Kubeflow local runner, LiteLLM as the model gateway to gpt-oss on Ollama, Grafana alerts and Jaeger.

Acceptance criteria

1. Sensitive fields never reach a chunk; dates and percentages do
2. Superseded and internal chunks are filtered by metadata; citation validity exactly 1.0, a prompt version on every answer
3. The prompt's store version matches `ANSWER_PROMPT.hash`, at stage `approved`
4. The vector store's count matches the manifest, and search returns the 2025 SLA first
5. The gate passes, fails when it should, and runs free on a pull request; `/drift` alerts after fifty off-topic questions

58 passed

what `make session04` prints when you are done

Questions worth asking

- You cut k from 8 to 2. Which metric moves first, and in which direction?
- The judge scores 0.5 on an answer you think is correct. Who is right, and how would you find out?
- Faithfulness rises on Monday after someone sharpened the prompt on Friday. Which three things in the report tell you it was the prompt?
- `/drift` alerts with the refusal rate at three times the baseline and a confidence PSI of 0.05. What moved?
- The store says one prompt version is approved and the manifest names another. Which one is serving?